FIAP ON

02

REST E E-MAIL COMO MOTORES DE PROCESSOS EM RPA

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Funcionamento do protocolo http.....13
Figura 2 - Exemplo de requisição.....15
Figura 3 - Exemplo de resposta 17

FIAP

LISTA DE QUADROS

Quadro 1 - Métodos Http.....19
Quadro 2 - Códigos de resposta http comumente utilizados.....21

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Consumo de serviços simples com Python 28
Código-fonte 2 - Uso de Método Get.....29
Código-fonte 3 - Uso de Método POST.....30
Código-fonte 4 - Uso de Método PUT 31
Código-fonte 5 - Uso de Método PATCH 32
Código-fonte 6 - Exemplo de método DELETE.....32
Código-fonte 7 - Consumo de APIs com tokens JWT 34
Código-fonte 8 - Exemplo de envio de e-mails com SMTP 40
Código-fonte 9 - Exemplo de recebimento de e-mails com POP3 43
Código-fonte 10 - Exemplo de recebimento de e-mails com IMAP 46
Código-fonte 11 - Exemplo de Caso Integrado 50

FIAP

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Consumo de serviço com curl 28

FIAP

SUMÁRIO

1 REST E E-MAIL COMO MOTORES DE PROCESSOS EM RPA 7
1.1 Introdução 7
2 API REST: FUNDAMENTOS E PRINCÍPIOS 9
2.2 O Protocolo HTTP 10
2.2.1 A origem e evolução do protocolo HTTP: motivações e primeiros usos 10
2.2.2 Funcionamento do protocolo http 12
2.2.3 Estrutura da requisição HTTP 13
2.2.4 Estrutura da resposta HTTP 15
2.2.5 Métodos HTTP 18
2.2.6 Códigos de resposta HTTP 19
3 DE VOLTA AO REST 23
3.1 Consumindo serviços REST ou REST APIs 25
3.1.1 Consumo de REST APIs em Python 26
3.1.2 Todas operações básicas 29
3.1.3 Controle de acesso a APIs com tokens JWT 33
3.1.4 Consumo de API com JWT 34
4 ENVIO E RECEBIMENTO DE E-MAILS COM SMTP, POP3 E IMAP: UMA ANÁLISE ESTRUTURADA DOS PROTOCOLOS DE CORREIO ELETRÔNICO 37
4.1 Arquitetura geral do sistema de e-mail 37
4.2 SMTP: Simple Mail Transfer Protocol 38
4.3 POP3: Post Office Protocol (Versão 3) 41
4.4 IMAP: Internet Message Access Protocol 44
4.5 Caso de uso automação de confirmação médica via e-mail: monitoramento IMAP com resposta SMTP 48
CONCLUSÃO 52
REFERÊNCIAS 54

1 REST E E-MAIL COMO MOTORES DE PROCESSOS EM RPA

1.1 Introdução

Vivemos em uma era na qual a informação tornou-se o recurso mais valioso para a automação de processos, a análise de dados e a tomada de decisões inteligentes. Contudo, embora a web contenha volumes colossais de dados acessíveis publicamente, nem sempre essas informações estão disponíveis de forma estruturada, padronizada ou amigável à leitura por máquinas. Diante disso, diversas estratégias de acesso programático à informação web emergiram como ferramentas indispensáveis para desenvolvedores, cientistas de dados e engenheiros de automação. Estas estratégias variam em robustez, formalismo e complexidade — e incluem, entre outras, o consumo de APIs RESTful, a prática de web scraping, a leitura de arquivos estruturados e até mesmo a exploração de feeds, endpoints ocultos e dados abertos governamentais.

O primeiro e mais robusto mecanismo de extração é o uso de APIs formais, projetadas justamente para permitir que os sistemas interajam com dados remotos de forma padronizada. APIs RESTful, GraphQL, SOAP e outras variantes oferecem pontos de acesso estáveis, geralmente documentados, que permitem operações específicas sobre recursos definidos, com respostas tipicamente no formato JSON ou XML. Essas APIs são consideradas o “caminho oficial” para a obtenção de dados de um sistema remoto e, por isso, são preferidas sempre que disponíveis. Para o consumidor, elas oferecem confiabilidade, versionamento, controle de acesso e previsibilidade — elementos fundamentais em sistemas corporativos ou produtos de software em larga escala.

Entretanto, existem inúmeros cenários nos quais APIs não estão disponíveis, seja por decisão estratégica do fornecedor dos dados, seja pela inexistência de uma camada de integração formal. Nestes casos, recorre-se à técnica conhecida como web scraping, que consiste na extração de dados diretamente a partir do conteúdo HTML das páginas da web. Embora essa prática seja menos formal e, por vezes, controversa do ponto de vista legal ou ético, ela continua sendo uma ferramenta vital para a automação de consultas, coleta de preços, extração de notícias, análise de concorrência e uma infinidade de outras aplicações. O scraping exige um olhar atento

à estrutura das páginas e ao uso de ferramentas como seletores CSS, expressões XPath e navegadores automatizados (exemplo: Selenium, Playwright) para acessar conteúdos dinâmicos.

Além das APIs e do scraping, existem outras formas de acessar dados na web. Alguns sites disponibilizam planilhas, arquivos CSV, JSON ou XML diretamente para download. Outros oferecem feeds RSS ou Atom, frequentemente usados por portais de notícias. Há ainda mecanismos de captura de informação que envolvem webhooks, integrações assíncronas, ou o uso de sistemas intermediários como serviços de ETL ou brokers de mensagens (Kafka, MQTT). Em todos os casos, o denominador comum é a capacidade de um agente humano ou automatizado interpretar os dados obtidos e integrá-los ao seu fluxo de trabalho.

É justamente sobre esses mecanismos de obtenção de dados que este capítulo se desenvolve. Embora o foco esteja na integração por meio de APIs RESTful seguras, será necessário compreender em profundidade como as URLs são estruturadas, como interpretar uma resposta JSON, e como manter sessões autenticadas com tokens como JWT. Ao longo das seções seguintes, o leitor aprenderá a operar como consumidor de serviços remotos, acessando dados com precisão, respeitando os protocolos esperados e extraindo valor a partir de fontes externas, de forma segura e replicável.

FIAP

2 API REST: FUNDAMENTOS E PRINCÍPIOS

Nos ecossistemas computacionais modernos, as APIs desempenham papel central como mecanismos de integração entre sistemas, permitindo que aplicações distintas — escritas em diferentes linguagens, executadas em diferentes ambientes e pertencentes a diferentes domínios — sejam capazes de se comunicar de maneira previsível, estruturada e sem acoplamento físico. Entre os estilos arquiteturais de APIs existentes, o mais difundido no contexto da web é o REST (Representational State Transfer), um modelo de arquitetura que se apoia nos princípios fundamentais do protocolo HTTP (Hypertext Transfer Protocol) e na ideia de que tudo em um sistema pode ser modelado como um recurso.

O termo REST foi introduzido por Roy Fielding em sua tese de doutorado no ano 2000, e rapidamente se consolidou como padrão de fato para a construção de sistemas escaláveis e interoperáveis. Seu sucesso reside na simplicidade e na aderência aos princípios que regem a própria web. Para o consumidor, isso representa um ganho enorme: ao aprender a consumir uma API REST, ele adquire competências transferíveis para dezenas, senão centenas, de outros serviços. Em outras palavras, o REST promove a padronização da comunicação entre cliente e servidor, mesmo em contextos heterogêneos.

Do ponto de vista do consumidor, a API REST se apresenta como uma coleção de URIs (identificadores de recursos) combinada com um conjunto limitado de operações semânticas baseadas nos verbos HTTP. Essas operações — GET, POST, PUT, DELETE, PATCH — indicam se o cliente deseja ler, criar, atualizar ou excluir um recurso. Os recursos em si são entidades do mundo real modeladas de forma conceitual, como clientes, produtos, pedidos, artigos, entre outros, e suas representações são transferidas geralmente no formato JSON, que oferece um bom equilíbrio entre legibilidade, leveza e suporte a estruturas aninhadas.

Tomemos como exemplo uma API de e-commerce. Nela, o recurso “produto” pode ser acessado por meio da URI https://api.loja.com/produtos. A requisição GET /produtos retorna a lista de todos os produtos, enquanto GET /produtos/42 retorna os detalhes de um produto específico. Se o consumidor deseja cadastrar um novo item, ele usa POST /produtos, enviando os dados no corpo da requisição. Para atualizá-lo,

envia PUT /produtos/42 e, para excluí-lo, usa DELETE /produtos/42. Essa lógica, por si só, já representa uma revolução: o cliente interage com conceitos claros e previsíveis, sem precisar conhecer os detalhes do banco de dados, da linguagem de backend ou da lógica de autenticação da aplicação.

A uniformidade dessa interface é intencional: ela simplifica o consumo, padroniza o comportamento e facilita a descoberta de novos endpoints. Do ponto de vista técnico, uma API REST bem projetada se comporta de maneira idempotente (especialmente nas requisições GET, PUT, DELETE), respeita códigos de status HTTP (como 200 OK, 404 Not Found, 400 Bad Request) e documenta claramente suas expectativas quanto ao conteúdo, formato e permissões.

Outro aspecto fundamental do REST é a separação entre cliente e servidor. O consumidor não precisa conhecer o estado interno da aplicação servidora, suas regras de negócio ou suas estruturas de dados — ele apenas manipula representações de recursos. Essa separação permite que o cliente e o servidor evoluam de forma independente. Em tempos de microsserviços e aplicações desacopladas, essa propriedade é especialmente valiosa, pois garante escalabilidade organizacional e tecnológica.

O REST não é uma tecnologia ou protocolo, mas sim um conjunto de restrições arquiteturais. A API que as adota pode ser implementada em qualquer linguagem, desde que respeite a semântica dos métodos HTTP, trate URIs como identificadores de recursos e ofereça representações que possam ser manipuladas pelo cliente. O consumidor, por sua vez, não precisa conhecer o REST profundamente. Basta que saiba identificar os endpoints, compreender os métodos HTTP e interpretar as respostas JSON. Esse é o foco do próximo tópico.

2.2 O Protocolo HTTP

2.2.1 A origem e evolução do protocolo HTTP: motivações e primeiros usos

O protocolo HTTP constitui a base da comunicação entre clientes e servidores na World Wide Web. Sua concepção remonta ao início da década de 1990, quando o físico e cientista da computação Tim Berners-Lee, então atuando no CERN

(Organização Europeia para a Pesquisa Nuclear), propôs um sistema distribuído de compartilhamento de documentos científicos. O objetivo central era permitir que pesquisadores ao redor do mundo acessassem informações técnicas de forma rápida, estruturada e interligada, superando os limites dos sistemas de documentação centralizados da época.

A proposta de Berners-Lee combinava três elementos fundamentais: um **protocolo de comunicação** (HTTP), um **identificador universal de recursos** (URL) e um **formato de documento com capacidade hipertextual** (HTML). O HTTP emergiu como a camada de aplicação responsável por viabilizar a troca de informações entre um navegador (cliente) e um servidor de conteúdo. Em sua primeira versão funcional, denominada HTTP/0.9, o protocolo era deliberadamente simples: suportava apenas o método GET, não utilizava cabeçalhos, não enviava metadados e cada conexão era encerrada após o envio de um único documento HTML.

Essa simplicidade, entretanto, era parte da sua força. Ao oferecer uma interface enxuta e de fácil implementação, o HTTP/0.9 possibilitou uma rápida adoção entre pesquisadores e desenvolvedores interessados em experimentar a nova arquitetura de hipertexto. Seu principal diferencial estava na capacidade de conectar documentos entre si por meio de hiperlinks, permitindo a navegação não linear — uma inovação conceitual que alteraria profundamente a maneira como o conhecimento seria organizado e disseminado em escala global.

À medida que a World Wide Web começou a ganhar visibilidade fora do meio acadêmico — especialmente com o lançamento de navegadores gráficos como Mosaic (1993) e Netscape Navigator (1994) —, tornou-se evidente a necessidade de ampliar as funcionalidades do protocolo. A transição para o HTTP/1.0 trouxe o suporte a cabeçalhos HTTP, codificação de conteúdo, tipos MIME e múltiplos métodos de requisição. Isso possibilitou a entrega de arquivos multimídia, a negociação de conteúdo entre cliente e servidor e o desenvolvimento de páginas dinâmicas com comportamento mais sofisticado.

A arquitetura cliente-servidor adotada pelo HTTP revelou-se especialmente adequada para aplicações distribuídas. Nessa configuração, o cliente (geralmente um navegador ou agente automatizado) envia uma requisição textual que especifica o recurso desejado, enquanto o servidor responde com uma mensagem contendo o conteúdo solicitado. Essa comunicação é sempre iniciada pelo cliente, e o protocolo,

por ser sem estado (*stateless*), não mantém memória de interações anteriores. Essa característica trouxe vantagens em termos de escalabilidade e paralelismo, ao mesmo tempo que exigiu o desenvolvimento de mecanismos complementares para persistência de sessões, como cookies e tokens.

É importante ressaltar que o HTTP foi concebido como um protocolo orientado a texto. Isso significa que todas as suas mensagens — tanto requisições quanto respostas — são formatadas em texto legível por humanos. Esse fator contribuiu significativamente para sua adoção em larga escala, uma vez que tornou o protocolo transparente e facilmente depurável. Os desenvolvedores podiam construir, testar e analisar requisições HTTP com ferramentas simples, como telnet, curl ou mesmo scripts personalizados, sem a necessidade de ambientes de desenvolvimento complexos.

Nos primeiros anos de uso, o HTTP foi utilizado primordialmente para a distribuição de páginas estáticas e documentos científicos. No entanto, seu modelo genérico de comunicação revelou-se versátil o suficiente para aplicações muito além da web tradicional. Com o tempo, o HTTP passou a ser a base de serviços de e-commerce, plataformas de e-mail, redes sociais e, mais recentemente, APIs RESTful e microsserviços em nuvem. Essa evolução foi acompanhada por melhorias nas versões subsequentes do protocolo, resultando nas versões HTTP/2 e HTTP/3, que introduziram recursos como multiplexação de conexões, compressão de cabeçalhos e uso de protocolos de transporte mais eficientes como o QUIC.

A história do HTTP é, portanto, indissociável da própria evolução da internet. Sua criação foi motivada por uma necessidade científica legítima: melhorar o acesso ao conhecimento e promover a colaboração distribuída. Contudo, sua arquitetura aberta, simples e extensível permitiu que o protocolo ultrapassasse as fronteiras do meio acadêmico e se tornasse a espinha dorsal da comunicação digital em escala global.

2.2.2 Funcionamento do protocolo http

O protocolo HTTP é a base da comunicação entre navegadores e servidores web. Ele define um conjunto de regras que permite que aplicações troquem

informações em formato estruturado, como texto, imagens ou dados de formulários. Sua arquitetura é baseada no modelo cliente-servidor, no qual o cliente inicia a requisição e o servidor responde com os dados solicitados. Essa troca acontece de forma simples, eficiente e padronizada, o que permitiu à Web crescer de forma escalável desde os anos 1990 até hoje.

Uma das características mais marcantes do HTTP é que ele não mantém memória entre uma requisição e outra. Cada requisição é independente, sem vínculo direto com as anteriores. Isso significa que o protocolo é sem estado, ou *stateless*. Por um lado, isso facilita a distribuição e o balanceamento de carga entre servidores, mas por outro, exige estratégias adicionais para manter sessões ou contexto, como o uso de cookies, tokens de autenticação ou armazenamento do lado do cliente.

Observe o fluxo ilustrado na figura a seguir:

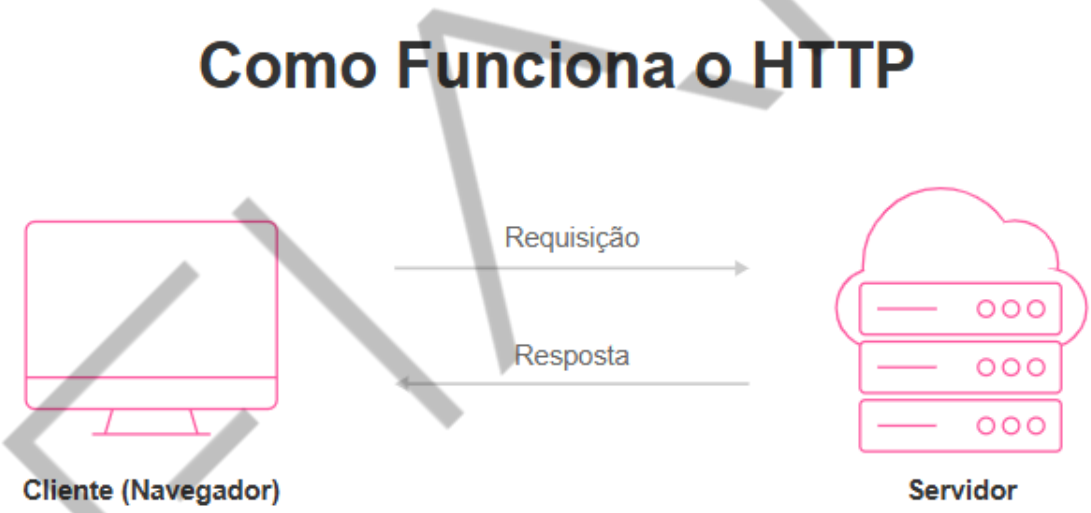


Figura 1 - Funcionamento do protocolo http
Fonte: Google Imagens (2025), adaptado por FIAP (2025)

2.2.3 Estrutura da requisição HTTP

A estrutura de uma requisição HTTP segue um formato textual padronizado e sequencial, dividido em três blocos principais: a **linha de requisição**, o **conjunto de cabeçalhos (headers)** e, opcionalmente, o **corpo (body)** da mensagem. Esses três elementos atuam em conjunto para definir tanto a intenção do cliente quanto os parâmetros da transação com o servidor. A **linha de requisição** é sempre a primeira linha do pedido, e sua estrutura é composta por três partes separadas por espaços: o

método HTTP (como GET, POST, PUT), o **caminho ou URI do recurso** desejado, e a **versão do protocolo**, como HTTP/1.1. Por exemplo, a linha POST /login HTTP/1.1 indica que o cliente deseja enviar dados ao recurso /login utilizando a versão 1.1 do protocolo HTTP.

Logo após a linha de requisição, encontram-se os cabeçalhos HTTP, que fornecem metadados essenciais para o processamento da requisição. Esses cabeçalhos seguem o formato **Nome: Valor** e são enviados um por linha, terminando com uma linha em branco que separa a seção de cabeçalhos do corpo da requisição. Entre os cabeçalhos mais comuns, destacam-se:

- **Host:** identifica o domínio da requisição; User-Agent, que descreve o agente cliente (navegador, app ou script);
- **Accept:** indica os formatos de resposta desejados (como application/json ou text/html);
- **Content-Type:** especifica o formato dos dados enviados;
- **Authorization:** responsável pela autenticação por tokens ou credenciais básicas.

Os **cabeçalhos** tornam a comunicação HTTP altamente configurável e extensível, permitindo que comportamentos variados sejam definidos sem alteração no protocolo em si.

A terceira e última parte de uma requisição HTTP é o **corpo da mensagem**, que é opcional e geralmente presente apenas nos métodos que envolvem envio de dados, como POST, PUT ou PATCH. O corpo contém os dados propriamente ditos que serão processados pelo servidor, como credenciais de login, formulários HTML, arquivos, ou representações de objetos em formatos como JSON ou XML. Por exemplo, em uma requisição POST para autenticação, o corpo pode conter um JSON com os campos "usuário" e "senha". A presença do corpo exige que o cabeçalho Content-Length ou Transfer-Encoding esteja presente, para que o servidor saiba quanto de dados esperar e como interpretar sua chegada. A figura a seguir ilustra o processo.

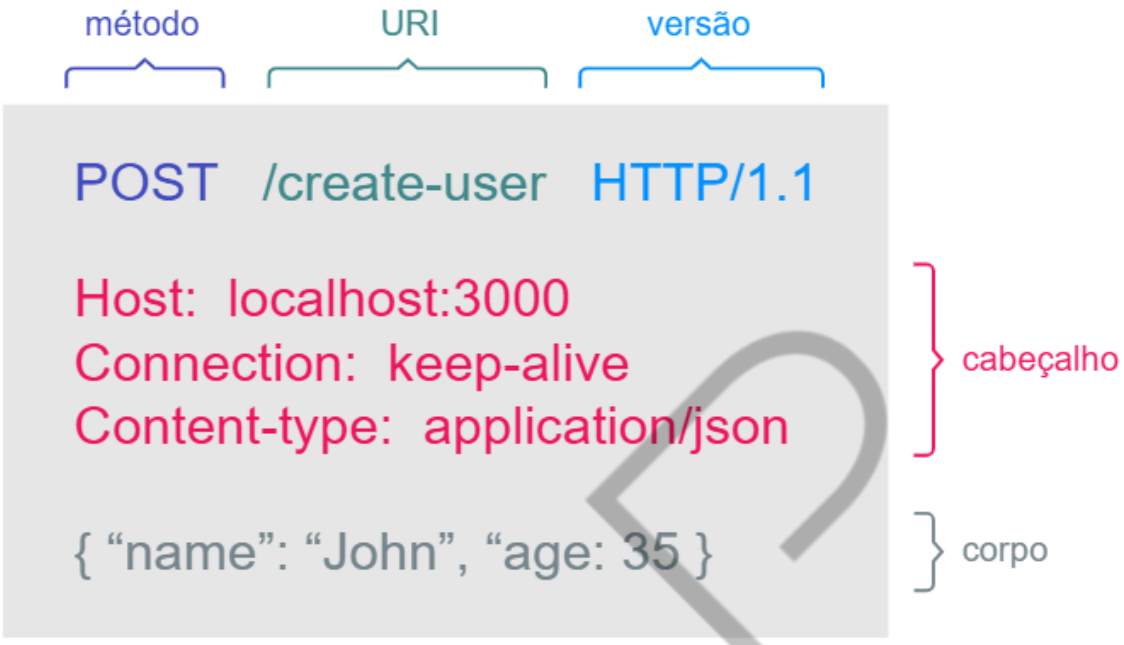


Figura 2 - Exemplo de requisição
Fonte: Google Imagens (2025), adaptado por FIAP (2025)

Conforme ilustrado na imagem da requisição HTTP, é possível visualizar claramente a divisão entre os três componentes principais. A linha de requisição aparece no topo, seguida por uma série de cabeçalhos com indentação vertical uniforme, encerrando com uma linha em branco. Logo abaixo, o corpo é apresentado como um bloco de texto separado, geralmente com estrutura JSON ou form-urlencoded. Essa organização visual reforça a natureza **estruturalmente modular e orientada a texto** do protocolo HTTP, permitindo que cada parte seja interpretada de forma independente pelo servidor. Essa legibilidade não apenas facilita o processo de debug e análise manual de requisições, como também simplifica o desenvolvimento de ferramentas automatizadas de inspeção, como sniffers de rede, consoles de navegador e clientes REST.

2.2.4 Estrutura da resposta HTTP

O servidor responde com uma estrutura semelhante. Ele envia uma linha de status, que indica se a requisição foi bem-sucedida, seguida de cabeçalhos que descrevem o conteúdo retornado e, frequentemente, um corpo com os dados solicitados. Esse corpo pode conter uma página HTML, um arquivo de imagem, um

documento PDF ou um payload estruturado como JSON, dependendo do contexto da aplicação.

A estrutura de uma resposta HTTP é organizada de forma similar à da requisição, mantendo o mesmo padrão de legibilidade e modularidade textual. No entanto, em vez de uma “linha de requisição”, a resposta inicia com uma **linha de status**, que tem a função de informar ao cliente o resultado da operação solicitada. Essa linha contém três elementos: a **versão do protocolo** (por exemplo, HTTP/1.1), um **código numérico de status** (como 200, 404 ou 500) e uma **mensagem textual de razão** que descreve brevemente o significado do código (como “OK”, “Not Found” ou “Internal Server Error”). Um exemplo clássico de linha de status seria HTTP/1.1 200 OK, indicando que a resposta foi bem-sucedida e que o conteúdo do recurso requisitado será retornado.

Logo após a linha de status, encontra-se a seção de cabeçalhos de resposta (response headers), que transmitem informações essenciais para o cliente interpretar o conteúdo da resposta. Esses cabeçalhos seguem a mesma convenção **Nome: Valor**, com um cabeçalho por linha, encerrados por uma linha em branco que precede o corpo da resposta. Entre os cabeçalhos mais frequentes, destacam-se:

- **Content-Type:** responsável por indicar o tipo de dado retornado (text/html, application/json, image/png, entre outros);
- **Content-Length:** que especifica o tamanho do corpo em bytes; Date, que informa o horário da geração da resposta;
- **Server:** responsável por identificar a tecnologia usada pelo servidor (como Apache, nginx ou Spring Boot).

Cabeçalhos adicionais como Cache-Control, ETag, Set-Cookie e Access-Control-Allow-Origin podem ser utilizados conforme o contexto e as políticas do servidor.

A seção seguinte é o corpo da resposta (body), onde se encontra o conteúdo principal retornado pelo servidor. A presença e o conteúdo do corpo variam de acordo com o tipo de requisição, o código de status e a configuração da aplicação. Em requisições bem-sucedidas do tipo GET, por exemplo, o corpo pode conter uma página HTML, um arquivo PDF, uma imagem ou uma estrutura JSON. Já em respostas com código 204 No Content, o corpo é ausente. Em respostas do tipo 500

Internal Server Error, o corpo pode conter mensagens de erro ou logs de exceção. O corpo da resposta é frequentemente interpretado por clientes web, APIs ou bibliotecas de rede e, por isso, é crucial que seu conteúdo esteja em conformidade com o tipo declarado no cabeçalho Content-Type.

A organização textual de uma resposta HTTP pode ser visualizada com clareza na figura a seguir, que ilustra os três componentes principais da resposta.



Figura 3 - Exemplo de resposta
Fonte: Google Imagens (2025), adaptado por FIAP (2025)

A linha de status aparece no topo, seguida por um bloco ordenado de cabeçalhos, encerrado por uma linha em branco. O corpo, quando presente, é exibido a seguir em seu formato nativo (HTML, JSON ou outro). A figura evidencia a natureza sequencial, legível e modular do protocolo, reforçando sua adequação a sistemas distribuídos e sua compatibilidade com ferramentas de inspeção e análise. Essa estrutura também torna a resposta HTTP altamente extensível e interpretável, permitindo a adição de novos cabeçalhos e formatos de conteúdo sem quebra de compatibilidade entre versões do protocolo ou entre agentes diferentes.

O HTTP é um protocolo baseado em texto, o que significa que suas mensagens podem ser facilmente lidas por humanos. Isso favorece a inspeção manual de requisições e respostas em ferramentas de depuração, como Postman, Insomnia ou o console de desenvolvedor dos navegadores. Também permite que desenvolvedores

criem e testem serviços de forma transparente, sem a necessidade de ferramentas especializadas nos estágios iniciais de desenvolvimento.

2.2.5 Métodos HTTP

O **HTTP (Hypertext Transfer Protocol)** é o protocolo base da comunicação na Web. Ele define como mensagens são formatadas e transmitidas entre clientes (normalmente navegadores ou frontends) e servidores (backends que hospedam recursos). Trata-se de um protocolo baseado em requisições e respostas, sem estado (*stateless*), operando geralmente sobre o protocolo de transporte TCP na porta 80 (ou HTTPS na 443, com camada de segurança). Um cliente realiza uma requisição HTTP enviando um comando (método), uma URL, cabeçalhos e, opcionalmente, um corpo com dados. O servidor interpreta essa requisição e responde com um código de status, cabeçalhos e, em muitos casos, também um corpo com dados.

Entre os **métodos HTTP**, o mais comum é o GET, que solicita a recuperação de um recurso identificado por uma URL. O GET não deve causar efeitos colaterais — ou seja, não deve modificar dados no servidor — e, por isso, é considerado seguro e idempotente (produz o mesmo resultado se executado várias vezes). Já o método POST é utilizado para envio de dados ao servidor, geralmente com a finalidade de criar novos recursos. Diferente do GET, o POST pode causar efeitos colaterais e não é idempotente.

Outro método importante é o PUT, que representa a substituição total de um recurso existente ou a criação de um novo recurso em um local específico. Ele é idempotente, o que significa que múltiplas chamadas consecutivas com os mesmos dados devem produzir o mesmo efeito. O PATCH, por sua vez, é utilizado para **atualizações parciais** de recursos — ele modifica apenas os campos indicados na requisição, sem substituir o recurso por completo, sendo útil para economizar banda e preservar campos inalterados.

O método DELETE é utilizado para remover um recurso identificado por uma URL. Assim como o PUT, ele é idempotente: deletar um mesmo recurso várias vezes resultará no mesmo estado final (recurso ausente). Já o HEAD é semelhante ao GET,

mas retorna apenas os cabeçalhos da resposta — é útil para verificar se um recurso existe ou inspecionar metadados, sem transferir seu conteúdo.

Além desses, o OPTIONS permite descobrir os métodos suportados por um recurso ou servidor, útil para negociações entre cliente e servidor, especialmente no contexto de CORS (Cross-Origin Resource Sharing). Os métodos TRACE e CONNECT são menos usados: o primeiro ecoa a requisição recebida para fins de depuração, enquanto o segundo estabelece túneis para conexões seguras, como em HTTPS. O quadro a seguir contém todos os métodos.

Método	**Descrição**
GET	Recupera dados de um recurso no servidor. Não deve alterar o estado.
POST	Envia dados para o servidor para criar um novo recurso.
PUT	Atualiza totalmente um recurso existente (ou cria, se não existir).
PATCH	Atualiza parcialmente um recurso existente.
DELETE	Remove um recurso do servidor.
HEAD	Igual ao GET, mas retorna apenas os cabeçalhos da resposta.
OPTIONS	Retorna os métodos HTTP suportados por um recurso.
TRACE	Retorna a requisição recebida (usado para diagnóstico).
CONNECT	Estabelece um túnel para comunicação segura (usado com HTTPS).

Quadro 1 - Métodos Http
Fonte: FIAP (2025)

2.2.6 Códigos de resposta HTTP

Quando o servidor responde a uma requisição, ele sempre retorna um **código de status HTTP**, que é uma indicação padronizada sobre o resultado da operação solicitada. Esses códigos são agrupados em cinco classes: 1xx (informativos), 2xx (sucesso), 3xx (redirecionamentos), 4xx (erros do cliente) e 5xx (erros do servidor). O código mais comum é o 200 OK, que significa que a requisição foi bem-sucedida e a resposta contém os dados solicitados. Outro muito utilizado é o 201 Created, que confirma a criação de um recurso com sucesso (por exemplo, após um POST).

Os códigos da classe 3xx indicam redirecionamentos. O 301 Moved Permanently informa que o recurso foi movido de forma permanente para outra URL, enquanto o 302 Found indica redirecionamento temporário. O código 304 Not Modified é utilizado em cenários de cache, informando ao cliente que o conteúdo requisitado

não mudou desde a última requisição (baseado nos cabeçalhos If-Modified-Since ou ETag) e, portanto, pode ser carregado localmente.

A categoria 4xx representa erros que se originam no cliente. O mais conhecido é o 404 Not Found, que indica que o recurso requisitado não foi encontrado no servidor. O 400 Bad Request significa que a requisição não pôde ser processada devido a erro de sintaxe ou parâmetros inválidos. Já o 401 Unauthorized e o 403 Forbidden lidam com falhas de autenticação e autorização, respectivamente: o primeiro implica ausência de credenciais válidas, enquanto o segundo significa que o acesso é proibido mesmo com credenciais.

Os códigos 5xx representam erros internos do servidor. O 500 Internal Server Error é uma resposta genérica para qualquer exceção não tratada no backend. Já o 502 Bad Gateway indica que o servidor atuando como gateway recebeu uma resposta inválida de outro servidor. O 503 Service Unavailable é comum em situações de manutenção ou sobrecarga, enquanto o 504 Gateway Timeout indica que o servidor não obteve resposta a tempo de outro serviço intermediário.

É importante ressaltar que os códigos de status e os métodos HTTP são componentes fundamentais para a construção de APIs RESTful robustas e previsíveis. A escolha correta dos métodos e o uso adequado dos códigos de resposta garantem clareza semântica nas interações entre frontend e backend, facilitando a manutenção, depuração e a documentação dos sistemas. Um resumo dos códigos e descrições de erros mais comuns estão no quadro a seguir:

Código	**Categoria**	**Significado**
200	Sucesso	OK – Requisição bem-sucedida.
201	Sucesso	Created – Recurso criado com sucesso.
202	Sucesso	Accepted – Requisição aceita, mas não finalizada.
204	Sucesso	No Content – Sem conteúdo de resposta.
301	Redirecionamento	Moved Permanently – Recurso movido.
302	Redirecionamento	Found – Redirecionamento temporário.
304	Redirecionamento	Not Modified – Conteúdo não modificado.
400	Erro do Cliente	Bad Request – Requisição malformada.
401	Erro do Cliente	Unauthorized – Requer autenticação.
403	Erro do Cliente	Forbidden – Acesso negado.
404	Erro do Cliente	Not Found – Recurso não encontrado.
405	Erro do Cliente	Method Not Allowed – Método não permitido.
409	Erro do Cliente	Conflict – Conflito no estado do recurso.

415	Erro do Cliente	Unsupported Media Type – Tipo de mídia não suportado.
429	Erro do Cliente	Too Many Requests – Limite de requisições excedido.
500	Erro do Servidor	Internal Server Error – Erro interno do servidor.
501	Erro do Servidor	Not Implemented – Método não implementado.
502	Erro do Servidor	Bad Gateway – Gateway inválido.
503	Erro do Servidor	Service Unavailable – Serviço indisponível.
504	Erro do Servidor	Gateway Timeout – Tempo de resposta esgotado.

Quadro 2 - Códigos de resposta http comumente utilizados
Fonte: FIAP (2025)

Vale destacar que o HTTP é um protocolo orientado a texto e altamente extensível. Cabeçalhos customizados podem ser definidos para transmitir informações específicas (como autenticação com tokens JWT, compressão com gzip, entre outros), e os corpos das mensagens podem utilizar formatos diversos como JSON, XML, HTML ou arquivos binários. Isso faz do HTTP uma ferramenta poderosa não apenas para navegação web, mas para integração entre serviços, Internet das Coisas (IoT), mobile apps e muito mais.

Ao longo dos anos, o protocolo passou por diversas atualizações para atender a demandas de desempenho e escalabilidade. A versão HTTP/1.1 trouxe melhorias como conexões persistentes e controle de cache mais avançado. Já o HTTP/2 introduziu recursos como multiplexação, compressão de cabeçalhos e envio de múltiplas requisições simultâneas na mesma conexão. Essas mudanças reduziram a latência e tornaram a navegação mais ágil, especialmente em aplicações complexas que carregam muitos recursos.

Mais recentemente, surgiu o HTTP/3, que substitui o protocolo TCP pelo QUIC, baseado em UDP. Essa mudança possibilita conexões mais rápidas e seguras, com menor impacto em redes instáveis, como as redes móveis. Além disso, o HTTP/3 melhora a segurança e a confiabilidade da comunicação, oferecendo melhor desempenho em aplicações modernas. Navegadores e servidores já vêm adotando essa nova versão progressivamente, tornando-a uma realidade nos grandes serviços online.

Outro aspecto importante do HTTP é sua extensibilidade. Ele permite a definição de cabeçalhos personalizados, negociação de formatos e autenticação de múltiplas formas. Essa flexibilidade tornou o protocolo útil não apenas para páginas web, mas também para APIs, aplicações móveis, IoT e serviços distribuídos em

nuvem. Sua simplicidade de base aliada à capacidade de extensão é o que torna o HTTP tão durável e adaptável em diferentes contextos tecnológicos.

FIAP

3 DE VOLTA AO REST

A arquitetura REST, ou Representational State Transfer, deve ser compreendida essencialmente como uma abstração arquitetural orientada à comunicação entre sistemas distribuídos baseada no protocolo HTTP. Diferentemente do que muitas vezes é divulgado em materiais introdutórios ou de natureza superficial, REST não é um protocolo, tampouco uma biblioteca ou tecnologia aplicável diretamente. Trata-se de um conjunto de restrições e princípios arquiteturais que, quando respeitados, permitem que aplicações interoperem de maneira escalável, previsível e desacoplada. Nesse contexto, sua implementação usual sobre HTTP é natural e eficaz, pois este último já fornece — de forma nativa — as estruturas que REST requer.

A base conceitual de REST se ancora na ideia de recursos — entidades logicamente identificáveis que representam dados ou comportamentos significativos em um sistema. Cada recurso é exposto por meio de uma URI (Uniform Resource Identifier) e manipulado através dos métodos oferecidos pelo protocolo HTTP. Assim, o método GET realiza a recuperação de um recurso; o POST, a criação de um novo recurso subordinado; o PUT, a substituição integral de um recurso existente; o PATCH, sua modificação parcial; e, por fim, o DELETE, sua exclusão. A correlação entre a semântica do protocolo HTTP e as operações típicas de manipulação de dados é direta e prescritiva, o que contribui para a uniformidade da interface — um dos pilares do modelo RESTful.

A construção de aplicações RESTful exige atenção cuidadosa ao princípio da ausência de estado. Isso significa que cada requisição HTTP enviada por um cliente deve conter todas as informações necessárias para que o servidor a compreenda e a processe de forma autônoma. Não há, portanto, qualquer garantia ou pressuposição de persistência de contexto entre requisições sucessivas. Essa restrição impõe certa redundância nas comunicações, mas em contrapartida, oferece grandes vantagens operacionais, como a facilidade de balanceamento de carga, a independência entre transações e a tolerância nativa a falhas. Em um ambiente distribuído e altamente concorrente, essas propriedades são determinantes para garantir desempenho e resiliência.

Outro aspecto essencial é a representação dos recursos. Ao contrário do que muitos sistemas legados implementam, o REST não transmite os próprios objetos do sistema, mas sim suas representações estruturadas, normalmente em formatos como JSON ou XML. É necessário que o servidor seja capaz de serializar o estado do recurso de forma padronizada, enquanto o cliente deve ser capaz de interpretá-lo com base no cabeçalho Content-Type. Esta separação clara entre a estrutura do recurso e sua representação facilita a independência entre cliente e servidor, permitindo evoluções assimétricas sem necessidade de interrupção ou coordenação explícita entre as partes.

A escolha pelo protocolo HTTP como suporte para a arquitetura REST não é aleatória. O HTTP já fornece nativamente muitos dos elementos exigidos: um modelo de requisição/resposta, métodos bem definidos com semântica clara, cabeçalhos para negociação de conteúdo, e status codes para representação precisa do resultado das operações. O REST apenas organiza essas capacidades em um modelo coeso e orientado a recursos, aproveitando ao máximo as propriedades do protocolo subjacente. Assim, a aplicação dos princípios REST sobre HTTP representa uma espécie de disciplina de uso, e não uma extensão ou modificação técnica do protocolo.

É comum que implementações modernas de APIs RESTful negligenciem certos princípios originalmente definidos, como o uso de hipermídia (HATEOAS) como mecanismo de transição de estado. Ainda assim, a adoção consistente dos métodos HTTP, da uniformidade de URIs e da ausência de estado já é suficiente para se beneficiar das qualidades fundamentais da arquitetura REST. Além disso, esse modelo se integra bem com mecanismos modernos de autenticação (como OAuth 2.0), versionamento, cache e monitoramento, tornando-se uma escolha adequada para serviços expostos em plataformas abertas, dispositivos móveis ou sistemas embarcados.

Portanto, entender REST não requer a introdução de novos paradigmas, mas sim a internalização das capacidades já oferecidas pelo HTTP e a adesão rigorosa a uma forma de organizar a comunicação entre sistemas. Quando você passa a reconhecer o REST como uma interpretação disciplinada do HTTP — e não como um componente externo —, compreende que o REST não propõe reinventar a comunicação em rede, mas sim racionalizá-la. Essa racionalização, baseada em

princípios sólidos de engenharia de software distribuído, é o que confere à arquitetura REST sua ampla adoção e longevidade.

Uma vez entendido o REST e, por sua vez, HTTP, vamos então consumir alguns serviços.

3.1 Consumindo serviços REST ou REST APIs

O consumo de APIs, em seu sentido mais abrangente, representa a prática pela qual agentes clientes interagem com serviços externos por meio de interfaces definidas programaticamente. Essas interfaces podem adotar estilos arquiteturais diversos, como REST, SOAP, GraphQL ou gRPC, cada um com suas convenções específicas de chamada, autenticação, formatação e negociação de dados. O ato de consumir uma API requer compreensão integral do seu contrato, o qual define os pontos de acesso, os parâmetros de entrada e saída, os formatos das mensagens, os mecanismos de segurança e os códigos de resposta previstos. A correta leitura e aplicação desse contrato é essencial para que haja integração funcional entre os sistemas envolvidos.

Quando se incorpora Inteligência Artificial (IA) ou Automação Robótica de Processos ao consumo de APIs, a natureza da comunicação deixa de ser meramente transacional e passa a assumir caráter decisório e adaptativo. Agentes baseados em IA podem consumir APIs para obter dados contextuais, realizar inferências, validar informações ou até construir respostas textuais por meio de modelos de linguagem. Já em fluxos de RPA, APIs são empregadas como pontos de acoplamento entre diferentes sistemas corporativos, permitindo a automação de tarefas antes realizadas manualmente. Um exemplo prático é a automação de processos financeiros que envolvem a consulta a uma API de crédito, a obtenção de documentos via API de armazenamento em nuvem e o registro de decisões em um ERP, tudo isso realizado por um agente de software integrado.

O consumo de APIs em ambientes com IA e RPA exige não apenas conhecimento técnico do estilo arquitetural utilizado, mas também maturidade na implementação de controle de falhas, segurança e adaptação semântica. É necessário lidar com a latência das respostas, a instabilidade dos serviços, o

versionamento das interfaces e as mudanças dinâmicas no domínio de dados. Ferramentas como mecanismos de retry com backoff exponencial, buffers intermediários, gateways de API e controle de acesso baseado em políticas tornam-se indispensáveis. Esses elementos precisam ser gerenciados de forma rigorosa para garantir que os agentes consumidores operem com confiabilidade e segurança em ambientes de missão crítica.

3.1.1 Consumo de REST APIs em Python

O consumo de APIs REST em Python é uma prática recorrente em projetos que demandam integração entre sistemas, coleta automatizada de dados, monitoramento em tempo real ou orquestração de serviços digitais. Essa abordagem é particularmente eficaz em ambientes que combinam Inteligência Artificial, automação de processos e sistemas distribuídos. O Python, sendo uma linguagem de alto nível voltada à produtividade e clareza sintática, oferece um ecossistema maduro de bibliotecas para comunicação via HTTP, atendendo a diferentes níveis de complexidade, controle e desempenho.

Entre as bibliotecas mais utilizadas, a requests ocupa uma posição de destaque. Seu principal diferencial está na simplicidade de uso aliada a uma API poderosa e flexível, que abstrai detalhes técnicos do protocolo HTTP sem sacrificar o controle necessário para tarefas mais sofisticadas. Com suporte completo a métodos verbais como GET, POST, PUT, PATCH e DELETE, a biblioteca permite o envio de dados em formatos variados, manipulação detalhada de cabeçalhos, cookies, autenticação e tratamento de códigos de resposta. Essa combinação a torna especialmente útil em aplicações corporativas, scripts de automação e agentes de decisão baseados em IA, proporcionando uma curva de aprendizado baixa e grande expressividade no código.

Para desenvolvedores que necessitam de controle mais próximo da camada de transporte, existem alternativas como http.client e urllib3, que fazem parte da biblioteca padrão ou de pacotes de suporte. Essas opções são mais verbosas e exigem um maior grau de detalhamento técnico, mas oferecem maior granularidade no manuseio de conexões, gerenciamento manual de redirecionamentos, persistência de sessão e configurações específicas de protocolo. Embora tecnicamente robustas,

essas bibliotecas são indicadas em contextos onde a simplicidade não é prioridade, como em sistemas embarcados, testes de carga ou diagnósticos de baixo nível.

Em aplicações que exigem execução assíncrona de múltiplas requisições simultâneas, como sistemas orientados a eventos ou agentes que operam sobre grandes volumes de chamadas concorrentes, bibliotecas como httpx e aiohttp representam escolhas eficientes. Ambas permitem a construção de aplicações não bloqueantes com o uso do paradigma async/await, otimizando o desempenho em arquiteturas de alta concorrência. Contudo, seu uso requer domínio adicional de programação assíncrona em Python, o que pode introduzir complexidade desnecessária em projetos que não demandam esse padrão.

Dado o escopo deste trabalho, que visa demonstrar boas práticas no consumo de APIs REST para fins de automação de processos e integração com componentes de IA, a biblioteca requests será adotada como padrão nas implementações. Essa decisão fundamenta-se em sua estabilidade, ampla adoção pela comunidade técnica, compatibilidade com bibliotecas auxiliares e alinhamento com os requisitos típicos de sistemas que consomem serviços remotos em contextos empresariais. Os exemplos apresentados a seguir ilustrarão, portanto, o uso da requests em situações práticas de consumo de APIs REST, permitindo replicação e adaptação direta por parte do leitor.

O Ministério da Saúde disponibiliza um portal de dados abertos que expõe, por meio de APIs REST, informações epidemiológicas, demográficas e administrativas de múltiplos programas de vigilância em saúde. A iniciativa integra a política federal de transparência pública, permitindo que pesquisadores, profissionais de TI e cidadãos obtenham conjuntos de dados atualizados em tempo quase real. Entre os recursos oferecidos, a coleção de arboviroses contém séries históricas sobre notificações de dengue, chikungunya e zika, organizadas em formato JSON e acessíveis por parâmetros de paginação padronizados, como limit e offset. O ponto de entrada principal da API encontra-se em https://apidadosabertos.saude.gov.br e pode ser consumido por qualquer cliente HTTP sem necessidade de autenticação prévia, bastando configurar o cabeçalho de aceitação ao tipo application/json.

Um acesso manual à coleção de casos de zika pode ser realizado de forma direta por meio da ferramenta curl, amplamente disponível em sistemas operacionais baseados em Unix e também instalável em plataformas Windows. O comando a seguir

demonstra a recuperação das primeiras cem ocorrências do recurso zikavirus na base de arboviroses, respeitando a semântica REST oferecida pela API.

```
curl -X 'GET' \

'https://apidadosabertos.saude.gov.br/arboviroses/zikavirus?li
mit=100&offset=0' \
       -H 'accept: application/json'
```

Comando de prompt 1 - Consumo de serviço com curl
Fonte: Elaborado pelo autor (2025)

Nesse comando, a opção -X 'GET' especifica o método HTTP utilizado, enquanto a URL completa contém o caminho do recurso e os parâmetros de paginação. O cabeçalho accept define que a resposta desejada deve vir no formato JSON. A execução retorna um objeto com metadados, total de resultados e a lista paginada de registros, possibilitando inspeção manual ou redirecionamento da saída para ferramentas de análise.

Para incorporar o mesmo acesso em um fluxo automatizado de análise ou integração de dados, utiliza-se a biblioteca requests em Python, selecionada por sua simplicidade e aderência às melhores práticas de consumo HTTP.

```
import requests
url =
"https://apidadosabertos.saude.gov.br/arboviroses/zikavirus"
params = {"limit": 100, "offset": 0}
headers = {"accept": "application/json"}

response = requests.get(url, params=params, headers=headers)
response.raise_for_status()
data = response.json()
print(f"Total de registros retornados:
{len(data['results'])}") print(data['results'][0])
```

Código-fonte 1 - Consumo de serviços simples com Python
Fonte: Elaborado pelo autor (2025)

A primeira instrução importa o módulo requests, responsável por realizar chamadas HTTP de alto nível. Em seguida, a variável url define o endpoint base do recurso de zika, enquanto o dicionário params contém os parâmetros de paginação limitados a 100 registros a partir do deslocamento zero. O dicionário headers explicita a preferência por respostas em JSON por meio da chave accept. A chamada requests.get executa a requisição GET, repassando URL, parâmetros e cabeçalhos. Em seguida, response.raise_for_status verifica se o código de estado HTTP indica

sucesso, lançando exceção em caso de erro. O método json desserializa a carga útil da resposta para um objeto Python. Por fim, o script imprime o número de elementos presentes no campo results e exibe o primeiro registro, demonstrando como acessar os dados retornados de forma programática.

3.1.2 Todas operações básicas

O serviço JSONPlaceholder é amplamente utilizado para testes e demonstrações de consumo de APIs REST, pois fornece endpoints que simulam operações CRUD completas sobre coleções de postagens, comentários, álbuns, fotos e usuários. Todos os recursos são acessíveis por HTTP, retornando dados em formato JSON sem exigir autenticação. A seguir, demonstramos as chamadas representativas para cada um dos métodos essenciais do protocolo HTTP, primeiro em cURL e, em seguida, em Python com a biblioteca requests, acompanhadas de explicação detalhada.

Operação GET: consulta de recurso

A operação GET é empregada para obter informações sobre um recurso previamente armazenado em um servidor. Em sistemas RESTful, essa operação é mapeada diretamente para ações de leitura, sem efeitos colaterais sobre o estado do servidor. Neste exemplo, será realizada a leitura de um recurso de postagem identificado pelo número 1. O servidor retorna um documento JSON com os campos correspondentes à entidade consultada.

```
import requests
url = "https://jsonplaceholder.typicode.com/posts/1" headers =
{"accept": "application/json"}

response = requests.get(url, headers=headers)
response.raise_for_status()
data = response.json()
print(data)
```

Código-fonte 2 - Uso de Método Get
Fonte: Elaborado pelo autor (2025)

Primeiramente, importa-se o módulo requests, responsável por realizar chamadas HTTP. A variável url define o endereço do recurso alvo da consulta. O

dicionário headers especifica que o cliente deseja receber a resposta em formato JSON, padrão nesta API. A função requests.get realiza a chamada HTTP do tipo GET, incluindo os cabeçalhos definidos. Em seguida, response.raise_for_status() verifica se a resposta retornou com código de sucesso (2xx), gerando exceção em caso de erro. A linha data = response.json() transforma a resposta em objeto nativo Python e, finalmente, print(data) exibe o conteúdo obtido.

Operação POST: criação de recurso

A operação POST é utilizada para criar um novo recurso no servidor. Neste contexto, a requisição inclui um corpo com os dados da nova entidade a ser criada. A API do JSONPlaceholder simula a criação de um recurso, retornando o objeto resultante com um identificador fictício.

```
import requests
import json

url = "https://jsonplaceholder.typicode.com/posts"
headers = { "accept": "application/json", "Content-Type":
"application/json" }
payload = {"title": "Novo post", "body": "Conteúdo de teste",
"userId": 7}
response = requests.post(url, headers=headers,
data=json.dumps(payload))
response.raise_for_status()
data = response.json()
print(data)
```

Código-fonte 3 - Uso de Método POST
Fonte: Elaborado pelo autor (2025)

A biblioteca json é importada para permitir a serialização do corpo da requisição. A variável url define o endpoint responsável pela criação de novos recursos. Os cabeçalhos indicam que a mensagem será enviada em JSON e que o cliente espera uma resposta no mesmo formato. O objeto payload representa a estrutura da nova postagem. O método json.dumps converte o objeto Python para JSON. A chamada requests.post envia os dados ao servidor. A verificação de sucesso é feita com raise_for_status, e o resultado é convertido em objeto nativo com response.json(), sendo então impresso.

Operação PUT: substituição completa de recurso

O método PUT substitui integralmente um recurso existente, exigindo que todos os seus campos sejam especificados. Utiliza-se aqui o mesmo identificador da operação GET, simulando a atualização completa do objeto de postagem.

```
import requests
import json

url = "https://jsonplaceholder.typicode.com/posts/1"
headers = { "accept": "application/json", "Content-Type":
"application/json" }

payload = { "id": 1, "title": "Título atualizado", "body":
"Conteúdo integralmente substituído", "userId": 7 }

response = requests.put(url, headers=headers,
data=json.dumps(payload))

response.raise_for_status()
data = response.json()
print(data)
```

Código-fonte 4 - Uso de Método PUT
Fonte: Elaborado pelo autor (2025)

A requisição é enviada ao mesmo recurso utilizado na consulta anterior. O corpo da mensagem especifica todos os atributos necessários para substituir o conteúdo anterior. A chamada requests.put realiza a substituição, com os cabeçalhos e dados serializados conforme exigido pela especificação. Após o processamento bem-sucedido, o conteúdo atualizado do recurso é retornado, desserializado e exibido.

Operação PATCH: atualização parcial de recurso

A operação PATCH permite alterar apenas parte de um recurso. É recomendada para atualizações em que apenas um subconjunto dos atributos será modificado, mantendo os demais inalterados.

```
import requests
import json

url = "https://jsonplaceholder.typicode.com/posts/1"
headers = {
     "accept": "application/json",
     "Content-Type": "application/json"
```

```
}

payload = {"title": "Título parcial"}

response = requests.patch(url, headers=headers,

data=json.dumps(payload))

response.raise_for_status()

data = response.json()

print(data)
```

Código-fonte 5 - Uso de Método PATCH
Fonte: Elaborado pelo autor (2025)

Diferentemente do PUT, o corpo da requisição PATCH contém apenas o campo a ser modificado. A chamada requests.patch envia essa modificação parcial ao servidor. O restante do processamento segue a mesma lógica dos exemplos anteriores: verificação do status HTTP, desserialização do corpo da resposta e exibição do resultado.

Operação DELETE: remoção de recurso

A operação DELETE é destinada à remoção de um recurso previamente identificado. Em APIs RESTful, essa operação representa uma exclusão lógica, física ou simbólica do recurso.

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"
headers = {"accept": "application/json"}

response = requests.delete(url, headers=headers)
response.raise_for_status()
print(f"Status de deleção: {response.status_code}")
```

Código-fonte 6 - Exemplo de método DELETE
Fonte: Elaborado pelo autor (2025)

A requisição é realizada ao recurso com identificador 1. O método requests.delete efetua a operação de exclusão. O código de status da resposta indica o sucesso da operação, sendo apresentado como confirmação final ao usuário. A API

JSONPlaceholder não realiza exclusão real, mas retorna código de sucesso simulado, ideal para testes de integração.

Cada um desses métodos ilustra uma das operações do ciclo CRUD (Create, Read, Update, Delete), apoiando-se sobre os princípios da arquitetura REST. As abordagens demonstradas são compatíveis com aplicações de automação, integração de sistemas, monitoramento de dados e acionamento de serviços cognitivos em fluxos de RPA.

3.1.3 Controle de acesso a APIs com tokens JWT

Em arquiteturas orientadas a serviços e sistemas distribuídos, o controle de acesso a recursos expostos por interfaces RESTful constitui uma preocupação fundamental. Diferente dos modelos baseados em sessões, que requerem persistência de estado no servidor, a autenticação baseada em **tokens** proporciona uma solução escalável e desacoplada. O padrão **JWT (JSON Web Token)** emergiu como uma resposta eficaz a essa demanda, oferecendo uma estrutura compacta e segura para transmissão de informações entre as partes autenticadas de um sistema. O JWT é amplamente utilizado em sistemas web modernos, plataformas móveis e aplicações que exigem controle granular de permissão por endpoint.

O JWT é composto por três seções: **header**, **payload** e **signature**. Todas codificadas em Base64URL e concatenadas por pontos. O **header** contém metadados sobre o token, como o algoritmo de assinatura (tipicamente HS256 ou RS256). O **payload** inclui declarações (claims), que são pares chave-valor que descrevem o usuário, permissões, escopos de acesso, tempo de expiração (exp), emissão (iat), entre outros. A **signature** é gerada a partir do header e do payload, utilizando a chave secreta ou par de chaves pública/privada. Essa assinatura garante a integridade do token e a confiabilidade do emissor.

A característica *stateless* do JWT significa que nenhuma informação adicional precisa ser armazenada no servidor após o login. O próprio token carrega consigo as credenciais e permissões, o que elimina a necessidade de sessões persistentes e facilita a escalabilidade horizontal da aplicação. Isso o torna ideal para arquiteturas baseadas em microsserviços e para integração com clientes diversos, como

navegadores, aplicativos móveis ou robôs de automação. A verificação de um token JWT pode ser feita de forma local e rápida, sem acesso a uma base de dados, desde que a chave de assinatura seja conhecida.

Outra vantagem relevante do JWT é sua interoperabilidade com outros padrões modernos de segurança, como **OAuth 2.0**, onde ele pode atuar como access token, e **OpenID Connect**, onde transporta a identidade do usuário. Em fluxos RPA (Robotic Process Automation), um robô autenticado com JWT pode realizar operações com total autonomia dentro do escopo definido, inclusive revalidando tokens expirados via refresh tokens, se o sistema o suportar. A utilização de tokens com curta duração e escopos restritos é uma boa prática que melhora a segurança e facilita a revogação seletiva.

Apesar de seu uso disseminado, o JWT exige cuidados. Por ser autoportante, qualquer exposição do token permite a apropriação indevida de identidade. Por isso, é fundamental trafegar o JWT exclusivamente por conexões seguras (HTTPS), definir tempos de expiração razoáveis (exp), implementar blacklist (em sistemas que exigem revogação), e jamais armazená-lo em locais acessíveis por scripts, como localStorage em navegadores. Com essas medidas, o JWT se consolida como uma das melhores práticas atuais em autenticação distribuída.

3.1.4 Consumo de API com JWT

```
import requests
import json

auth_url = "https://api.exemplo.com/auth/login"
data_url = https://api.exemplo.com/dados/cliente
credentials = {"email": "usuario@dominio.com", "senha":
"senha123"}

auth_response = requests.post(auth_url, json=credentials)
auth_response.raise_for_status()
token = auth_response.json().get("access_token")
headers = { "Authorization": f"Bearer {token}", "Accept":
"application/json" }
data_response = requests.get(data_url, headers=headers)
data_response.raise_for_status()
print(data_response.json())
```

Código-fonte 7 - Consumo de APIs com tokens JWT
Fonte: Elaborado pelo autor (2025)

O código anterior demonstra, de forma sintética e funcional, o fluxo de autenticação baseado em JWT aplicado ao consumo de uma API REST. Primeiramente, o cliente realiza uma requisição HTTP POST ao endpoint de login (auth_url), enviando suas credenciais no corpo da requisição. O servidor valida essas credenciais e, em caso de sucesso, devolve um token JWT, geralmente no campo "access_token" do corpo da resposta. Esse token é então extraído para ser reutilizado nas chamadas subsequentes.

Em seguida, monta-se o cabeçalho de autorização no formato Bearer, conforme exigido pela maioria das APIs REST que utilizam JWT. Esse cabeçalho é então inserido em uma nova requisição GET direcionada ao recurso protegido (data_url). O servidor verifica a validade e autenticidade do token antes de responder com os dados requisitados. A chamada raise_for_status() garante a interrupção imediata do processo em caso de erro, enquanto o método json() permite extrair e manipular os dados da resposta de forma estruturada.

Na prática, este fluxo é facilmente integrável a sistemas de automação baseados em Python, incluindo RPA e bots inteligentes. A obtenção, renovação e uso dos tokens pode ser incorporada como parte de rotinas agendadas, pipelines de integração contínua, ou módulos de coleta e análise de dados. A segurança, a portabilidade e a independência de estado que o JWT proporciona tornam-no um padrão robusto e moderno, compatível com os desafios da autenticação segura em ambientes altamente distribuídos e heterogêneos.

O consumo de APIs REST com autenticação baseada em tokens, especialmente no padrão JWT, representa um alicerce estratégico para a construção de soluções inteligentes em RPA e aplicações assistidas por IA. A capacidade de integrar sistemas distintos por meio de interfaces bem definidas permite que os agentes autônomos acessem, manipulem e interpretem dados em tempo real, reduzindo a dependência de intervenções humanas e promovendo fluxos de trabalho mais ágeis, auditáveis e escaláveis. Ao incorporar lógica de decisão baseada em IA sobre dados obtidos via APIs, torna-se possível automatizar operações complexas como validação de documentos, categorização de conteúdos ou acionamento de regras condicionais sem a necessidade de interação direta com as interfaces visuais tradicionais.

Disponibilizamos um código para que você possa ir além e simular uma API REST. Você encontrará esse código na seção Assets desta fase, chamado “jwt_service_consumer.py”. Basta executar esse Python em seu VSCode via botão Play e deixar que o VSCode instale as instâncias caso sejam necessárias.

Para simular serviços com tokens utilizando JWT, deixamos outros dois Assets chamados “servidor.py” e “cliente.py” também nessa fase.

Encerrando esta seção dedicada à comunicação entre agentes inteligentes e servidores via REST, avançamos agora para um novo componente essencial em soluções automatizadas: o envio e recebimento de e-mails por meio dos protocolos SMTP, POP3 e IMAP. Em contextos corporativos, o correio eletrônico ainda é um dos principais canais de acionamento de processos operacionais e tomada de decisão. Agentes de RPA e sistemas baseados em IA podem monitorar caixas postais, identificar padrões semânticos nas mensagens e atuar diretamente a partir desses estímulos, seja para iniciar um processo interno, responder automaticamente a uma solicitação ou alimentar outros sistemas a partir de informações extraídas. A seguir, exploraremos essas possibilidades com profundidade técnica e exemplos práticos voltados à automação inteligente de comunicações eletrônicas.

FIAP

4 ENVIO E RECEBIMENTO DE E-MAILS COM SMTP, POP3 E IMAP: UMA ANÁLISE ESTRUTURADA DOS PROTOCOLOS DE CORREIO ELETRÔNICO

O correio eletrônico, ou e-mail, é uma das tecnologias mais duradouras e fundamentais da internet moderna. Mesmo diante da proliferação de sistemas de mensagens instantâneas e redes sociais, o e-mail permanece como um meio oficial, documentado e assíncrono de comunicação. Sua robustez, confiabilidade e compatibilidade entre sistemas heterogêneos decorrem da adoção de um conjunto padronizado de protocolos de comunicação.

Entre esses protocolos, destacam-se o SMTP (Simple Mail Transfer Protocol), utilizado para envio de mensagens, e os protocolos POP3 (Post Office Protocol, versão 3) e IMAP (Internet Message Access Protocol), destinados ao recebimento e gerenciamento das mensagens no lado do cliente. Cada um desses protocolos possui uma arquitetura, semântica e limitações próprias, sendo escolhidos conforme o tipo de aplicação, a topologia da rede e o perfil do usuário.

Compreender a interação entre esses protocolos é essencial para profissionais de redes, desenvolvedores de sistemas distribuídos e especialistas em segurança da informação. Este tópico visa analisar detalhadamente a estrutura e o funcionamento dos protocolos SMTP, POP3 e IMAP, fornecendo exemplos práticos e comparações técnicas.

A abordagem proposta adota uma perspectiva tecno-científica, apresentando sistematicamente os comandos, fluxos de execução, exemplos de sessões e implicações de segurança. O objetivo é não apenas descrever o funcionamento dos protocolos, mas também discutir suas motivações históricas, limitações conceituais e papel atual no ecossistema digital.

4.1 Arquitetura geral do sistema de e-mail

A comunicação via e-mail é baseada em uma arquitetura cliente-servidor composta por três elementos essenciais: o cliente de e-mail, o servidor de envio e o servidor de recebimento. O cliente é o software ou aplicação utilizado pelo usuário

final para compor, enviar, ler e organizar mensagens. Os servidores, por sua vez, desempenham o papel de retransmitir (SMTP) e armazenar (POP3/IMAP) essas mensagens.

No processo de envio, o cliente utiliza o protocolo SMTP para transmitir a mensagem ao servidor de envio do domínio do remetente. Esse servidor, então, consulta o DNS para localizar o servidor de destino e repassa a mensagem via SMTP. Já no processo de recebimento, o servidor do destinatário armazena a mensagem até que o cliente do usuário a acesse via POP3 ou IMAP.

A interação entre esses componentes é orquestrada por mecanismos de endereçamento eletrônico (endereços de e-mail, MX records), autenticação de usuários, protocolos de transporte (TCP/IP) e padrões de formatação de mensagens (como o MIME).

Atenção: o correto funcionamento desse ecossistema depende de configurações coerentes em ambos os lados da comunicação.

Com a crescente demanda por segurança e mobilidade, surgiram extensões e variações dos protocolos originais, como SMTP com STARTTLS e autenticação SASL, bem como IMAP com suporte a push via IDLE. Tais melhorias têm permitido a continuidade do e-mail como ferramenta central de comunicação organizacional e pessoal.

4.2 SMTP: Simple Mail Transfer Protocol

O Simple Mail Transfer Protocol é o padrão para envio de mensagens eletrônicas através de redes IP. Definido originalmente na RFC 821, e atualizado posteriormente pela RFC 5321, o SMTP opera como protocolo da camada de aplicação sobre a camada de transporte TCP, utilizando normalmente as portas 25, 587 (com autenticação) e 465 (com SSL/TLS implícito). O SMTP define regras para transmissão de mensagens ponto a ponto entre servidores, bem como o envio inicial do cliente para o servidor de saída.

Seu funcionamento baseia-se em um diálogo textual do tipo comando-resposta, em que o cliente emite comandos como HELO/EHLO, MAIL FROM, RCPT TO, DATA

e QUIT, e o servidor responde com códigos numéricos que indicam o sucesso, erro temporário ou falha permanente na operação. Esse modelo torna o protocolo transparente e facilmente depurável, sendo amplamente utilizado por desenvolvedores e administradores de sistemas.

Uma das principais limitações do SMTP é sua falta de segurança nativa. O protocolo original não possui criptografia nem autenticação, o que permitiu o surgimento de práticas maliciosas como o spoofing de remetentes e o envio massivo de spam. Como forma de mitigar esses problemas, foram incorporadas extensões como o STARTTLS, que permite a negociação de criptografia, e mecanismos de autenticação como o SMTP AUTH, que exige credenciais antes do envio.

No processo de entrega, é comum que o servidor de origem utilize uma fila de mensagens e tente retransmitir a comunicação com o servidor de destino várias vezes em caso de falhas temporárias. Quando a entrega é bem-sucedida, a mensagem é armazenada no servidor de destino, aguardando leitura pelo usuário final. Caso contrário, é enviada uma notificação de falha ao remetente original.

Para ilustrar melhor, imagine uma empresa que precisa enviar diariamente relatórios automáticos a diversos destinatários — internos e externos. Esse processo é comumente encontrado em setores como contabilidade, recursos humanos e gestão de projetos. Automatizar o envio desses e-mails elimina tarefas repetitivas, reduz erros manuais e assegura a pontualidade na comunicação.

A biblioteca smtplib, integrada ao Python, permite esse tipo de automação. Para enriquecer o conteúdo da mensagem com HTML, anexos ou cabeçalhos personalizados, pode-se utilizar também o módulo email.message. Quando o envio é feito por servidores que exigem conexão segura, recomenda-se o uso da biblioteca ssl, que também está inclusa na biblioteca padrão.

```
     import smtplib, ssl
     from email.message import EmailMessage

     # Cria uma nova mensagem de e-mail
     msg = EmailMessage()
     msg.set_content("Olá, este é o relatório automático do
dia.") # Define o corpo da mensagem




     #Define o assunto do e-mail
     msg["Subject"] = "Relatório Diário" #Assunto
     msg["From"] = "automacao@empresa.com" # Remetente

     msg["To"] = "gerente@empresa.com" # Destinatário

     # Cria contexto SSL para comunicação segura
     context = ssl.create_default_context() ]
     with smtplib.SMTP_SSL("smtp.empresa.com", 465,
context=context) as server: # Conecta ao servidor SMTP com SSL

     # Realiza login na conta de e-mail
     server.login("automacao@empresa.com", "SUA_SENHA_SEGURA_")
     server.send_message(msg) # Envia a mensagem
```

Código-fonte 8 - Exemplo de envio de e-mails com SMTP
Fonte: Elaborado pelo autor (2025)

O trecho de código apresentado tem por finalidade realizar o envio automatizado de um e-mail utilizando o protocolo SMTP sobre uma conexão segura via SSL/TLS. O processo inicia-se com a importação dos módulos smtplib, ssl e email.message.EmailMessage. O módulo smtplib provê a interface para comunicação com servidores SMTP, enquanto ssl é responsável por estabelecer o contexto de segurança da conexão. A classe EmailMessage permite a construção estruturada da mensagem, segundo os padrões MIME, o que facilita a definição de corpo textual e cabeçalhos da mensagem de forma clara e segura.

A seguir, é instanciado um objeto EmailMessage, e o conteúdo principal do e-mail é adicionado com o método set_content(), especificando uma mensagem em texto plano. Em seguida, são atribuídos três campos essenciais: Subject, que define o assunto da mensagem; From, que indica o endereço de e-mail do remetente; e To, que define o destinatário. Esses metadados compõem os cabeçalhos da mensagem e são fundamentais para o correto encaminhamento e exibição nos clientes de e-mail.

Posteriormente, é criado um contexto de segurança com ssl.create_default_context(), que configura os parâmetros necessários para uma conexão segura, como certificados confiáveis e versões do protocolo TLS suportadas. A conexão ao servidor de e-mail é então realizada por meio do bloco with, utilizando smtplib.SMTP_SSL, que estabelece uma sessão segura na porta 465 — convencionalmente utilizada para SMTP com SSL implícito. O uso do bloco with garante que a conexão será corretamente encerrada ao final da operação.

No interior deste bloco, realiza-se a autenticação junto ao servidor por meio do método login(), que recebe o e-mail do remetente e sua respectiva senha. Uma vez autenticado, o método send_message() é invocado para enviar a mensagem configurada anteriormente. Dessa forma, o e-mail é transmitido de forma segura e automatizada ao servidor SMTP, que se encarrega de encaminhá-lo ao destinatário especificado.

Esse script pode ser incorporado a um pipeline maior, integrado a scripts de geração de relatórios, e programado com cron ou agendadores como APScheduler. A automação garante não apenas consistência na operação, mas também rastreabilidade e auditoria das comunicações.

4.3 POP3: Post Office Protocol (Versão 3)

O Post Office Protocol na sua terceira versão, conhecido como POP3, é um protocolo de acesso remoto a mensagens de e-mail armazenadas em um servidor. Projetado para ser simples e eficiente, o POP3 permite ao cliente baixar mensagens e armazená-las localmente, comumente removendo-as do servidor após o processo. Essa abordagem, embora eficaz para acesso offline, apresenta limitações para sincronização entre múltiplos dispositivos.

O POP3 opera sobre o protocolo TCP, utilizando a porta 110 para conexões não criptografadas e a porta 995 para conexões seguras com SSL/TLS. Sua estrutura baseia-se em comandos textuais simples como USER, PASS, STAT, LIST, RETR, DELE e QUIT, permitindo autenticação do usuário, listagem de mensagens, recuperação individual e remoção de e-mails da caixa postal do servidor.

O protocolo segue três fases bem definidas: **autenticação**, **transação** e **atualização**. Durante a autenticação, o cliente fornece as credenciais de acesso ao servidor. Na fase de transação, o cliente pode manipular as mensagens, baixando e marcando aquelas que deseja excluir. Por fim, na fase de atualização, ao enviar o comando QUIT, o servidor realiza a remoção definitiva das mensagens marcadas para exclusão.

Embora limitado em termos de funcionalidade, o POP3 ainda é amplamente utilizado em cenários com conectividade restrita, onde os usuários preferem manter cópias locais dos e-mails. Sua simplicidade o torna leve e de fácil implementação, especialmente em dispositivos embarcados ou clientes minimalistas. No entanto, em ambientes que demandam acesso simultâneo e sincronização contínua entre dispositivos, o POP3 é frequentemente substituído pelo IMAP.

No contexto de automação de processos, um caso comum é aguardar um e-mail que contenha uma solicitação de processo específico — por exemplo, iniciar o processamento de um pedido recebido. A seguir, apresentamos um exemplo onde um sistema monitora mensagens com remetente autorizado ou com assunto iniciando por um prefixo específico (como "Pedido:").

```
import poplib from email.parser
import BytesParser

pop3_server = poplib.POP3_SSL('pop.empresa.com')
pop3_server.user('suporte@empresa.com') # Informa usuário
pop3_server.pass_('senha_suporte') # Informa senha

num_msgs = len(pop3_server.list()[1])

num_msgs = len(pop3_server.list()[1]) # Obtém número de
mensagens

for i in range(num_msgs):
     raw_email = b"\n".join(pop3_server.retr(i+1)[1])
     email_msg = BytesParser().parsebytes(raw_email)
     remetente = email_msg.get("From", "")
     assunto = email_msg.get("Subject", "")

     if "cliente@empresa.com" in remetente or
assunto.startswith("Pedido:"):

     print("Processo iniciado para:", assunto)
```

```
    # Aqui o código iniciaria o processo de negócio com base
no e-mail recebido
    invocarProcessoDeNegocio()

pop3_server.quit()
```

Código-fonte 9 - Exemplo de recebimento de e-mails com POP3
Fonte: Elaborado pelo autor (2025)

O código apresentado exemplifica o uso do protocolo **POP3 (Post Office Protocol version 3)** em um ambiente seguro com **SSL (Secure Sockets Layer)**, para a **leitura automatizada de mensagens de e-mail** a partir de uma caixa postal remota. Tal abordagem é comum em cenários de **automação de processos**, onde decisões ou execuções de tarefas são disparadas com base em mensagens recebidas.

A implementação inicia-se com a importação dos módulos poplib e BytesParser, pertencente ao submódulo email.parser. O primeiro provê a funcionalidade de conexão com servidores POP3, autenticação de usuários e recuperação de mensagens, enquanto o segundo permite analisar e estruturar e-mails no formato MIME, a partir de seus dados brutos em bytes.

A conexão com o servidor remoto é realizada por meio da chamada poplib.POP3_SSL('pop.empresa.com'), a qual estabelece um canal seguro com o servidor POP3 utilizando a porta padrão 995. Em seguida, são fornecidas as credenciais de autenticação por meio dos métodos user() e pass_(), que garantem o acesso autorizado à caixa de entrada do endereço de e-mail corporativo configurado.

Com a conexão autenticada, o código executa pop3_server.list() para obter a lista de mensagens disponíveis. Essa função retorna uma lista de descritores das mensagens, cuja quantidade é calculada pela função len(), atribuindo-se o resultado à variável num_msgs. Esse valor serve como base para iteração sequencial sobre todas as mensagens disponíveis no servidor.

Dentro do laço for, cada mensagem é recuperada individualmente por meio da chamada pop3_server.retr(i+1), que retorna o conteúdo da mensagem na forma de uma lista de bytes (linha a linha). Essa lista é unificada com b"\n".join(...), resultando em uma string binária única representando o e-mail completo. Em seguida, essa string bruta é transformada em um objeto estruturado utilizando BytesParser().parsebytes(raw_email), o qual permite o acesso direto aos campos MIME da mensagem, como remetente (From) e assunto (Subject).

A lógica condicional incorporada na sequência representa um típico gatilho de automação: o sistema verifica se o remetente do e-mail é "cliente@empresa.com" ou se o campo de assunto inicia com o prefixo "Pedido:". Se qualquer uma dessas condições for satisfeita, considera-se que a mensagem contém uma solicitação de ação, e o sistema imprime uma notificação no console indicando que um processo será iniciado. Nesse ponto, o código pode ser expandido para acionar rotinas específicas, como integração com sistemas de ERP, abertura de chamados, ou execução de scripts de workflow.

Por fim, a conexão com o servidor é encerrada adequadamente com pop3_server.quit(), garantindo a liberação dos recursos da sessão estabelecida.

Este exemplo demonstra uma aplicação robusta e segura de recepção e inspeção de mensagens de e-mail com base no protocolo POP3, reforçando seu uso em **sistemas reativos orientados por eventos** provenientes de mensagens eletrônicas. O uso de SSL assegura a confidencialidade e integridade das comunicações, enquanto o parser MIME permite que os dados sejam tratados de forma estruturada e confiável no ambiente da aplicação.

Essa abordagem viabiliza a integração entre sistemas de e-mail e ferramentas de RPA, atuando como um gatilho de processos baseado na chegada de mensagens com critérios específicos. É particularmente útil para sistemas de pedidos, atualizações de status e workflow de atendimento ao cliente.

Deixamos um Hands On para que você possa testar e simular o envio de e-mails para você mesmo. Para isso, basta executar no VSCode o Assets chamado “mail_sender.py” que está disponível aqui nesta fase.

4.4 IMAP: Internet Message Access Protocol

O Internet Message Access Protocol representa uma abordagem mais sofisticada e moderna para o acesso a mensagens de e-mail. Diferentemente do POP3, o IMAP foi projetado desde sua concepção para suportar múltiplos dispositivos acessando simultaneamente a mesma caixa de correio, mantendo o estado das mensagens sincronizado entre o servidor e os clientes. Essa característica o torna

especialmente adequado para o contexto atual de mobilidade e uso de dispositivos diversos.

O IMAP opera geralmente sobre a porta 143 para conexões não criptografadas e a porta 993 para conexões seguras com SSL/TLS. Seus comandos são mais extensos e estruturados que os do POP3, permitindo ações como listar pastas (LIST), selecionar caixas de mensagens (SELECT), buscar mensagens com critérios complexos (SEARCH), marcar mensagens com flags (STORE) e manter conexões persistentes via comandos como IDLE, que habilitam notificações em tempo real.

Uma das principais vantagens do IMAP está na sua capacidade de gerenciamento remoto da caixa de correio. As mensagens permanecem no servidor por padrão, e qualquer ação realizada em um cliente, como leitura, exclusão ou organização em pastas, é refletida nos demais clientes conectados. Esse comportamento garante consistência e facilita o uso colaborativo e contínuo da conta de e-mail.

O IMAP ainda é compatível com mecanismos modernos de autenticação e criptografia, suportando autenticação SASL, STARTTLS e mecanismos OAuth2 em ambientes mais complexos. Seu modelo de operação favorece a escalabilidade de serviços de webmail, clientes móveis e integrações corporativas, consolidando-se como o protocolo preferencial em serviços de e-mail em nuvem como Gmail, Outlook e Yahoo Mail.

Suponha que o sistema precise ficar monitorando uma caixa de entrada aguardando um e-mail com um prefixo no assunto como "Autorizar:" ou proveniente de um remetente específico como "diretoria@empresa.com". O objetivo é executar automaticamente uma liberação de tarefa, disparo de fatura ou resposta automática.

IMAP

```
import imaplib, email

imap = imaplib.IMAP4_SSL("imap.empresa.com")
imap.login("automacao@empresa.com", "senha_automacao")
imap.select("inbox") status, messages = imap.search(None,
'ALL')

for num in messages[0].split():
     status, data = imap.fetch(num, '(RFC822)')
     raw_email = data[0][1]
     msg = email.message_from_bytes(raw_email)
     remetente = msg.get("From", "")
     assunto = msg.get("Subject", "")
     if "diretoria@empresa.com" in remetente or
assunto.startswith("Autorizar:"):

        print("Processo autorizado com base na mensagem:",
assunto)

imap.logout()
```

Código-fonte 10 - Exemplo de recebimento de e-mails com IMAP
Fonte: Elaborado pelo autor (2025)

O código apresentado demonstra uma aplicação prática do protocolo **IMAP** para leitura automatizada de mensagens eletrônicas a partir de uma caixa de entrada remota. Ao contrário do protocolo POP3, que realiza o download e a eventual exclusão de mensagens do servidor, o IMAP permite **acesso não destrutivo e gerenciado às mensagens**, possibilitando que múltiplos clientes consultem a mesma caixa postal de forma sincronizada. Essa característica torna o IMAP particularmente apropriado para sistemas automatizados de triagem, supervisão ou aprovação de mensagens.

A implementação utiliza os módulos imaplib e email, ambos da biblioteca padrão do Python. O primeiro oferece os recursos necessários para estabelecer conexões seguras com servidores IMAP e manipular caixas de correio remoto. O segundo, por sua vez, fornece as ferramentas para decodificar e estruturar mensagens MIME, permitindo o acesso aos campos de metadados e ao conteúdo da mensagem.

A conexão com o servidor é realizada por meio da chamada imaplib.IMAP4_SSL(...), que estabelece um canal seguro via TLS/SSL, neste caso com o servidor de e-mails identificado pelo domínio "imap.empresa.com". Em seguida,

o método login(...) realiza a autenticação do usuário, garantindo que apenas contas autorizadas tenham acesso à caixa postal especificada.

Após autenticação bem-sucedida, o comando select("inbox") define que as operações subsequentes serão realizadas sobre a **caixa de entrada principal**. Em seguida, imap.search(None, 'ALL') solicita ao servidor a listagem de todas as mensagens disponíveis na pasta corrente, retornando seus identificadores numéricos. O processamento principal do código ocorre dentro de um laço for, que percorre esses identificadores.

Para cada mensagem listada, executa-se imap.fetch(num, '(RFC822)'), que requisita o conteúdo completo da mensagem no formato bruto conforme a especificação **RFC 822**, padrão que descreve a estrutura dos e-mails em texto plano. O conteúdo bruto da mensagem, representado como uma sequência de bytes, é convertido em um objeto estruturado com email.message_from_bytes(...), permitindo o acesso a campos específicos como o remetente (From) e o assunto (Subject).

O sistema então aplica uma **lógica condicional automatizada**, inspecionando se a mensagem foi enviada por "diretoria@empresa.com" ou se o assunto começa com a palavra-chave "Autorizar:". Essa abordagem representa um modelo reativo clássico, em que o sistema interpreta o e-mail como um **gatilho de autorização**. Ao detectar uma das condições configuradas, o programa imprime no console uma mensagem indicando que o processo está autorizado, podendo ser expandido para executar rotinas automatizadas de liberação, aprovação ou integração com sistemas administrativos.

Finalmente, o comando imap.logout() finaliza a sessão de forma segura, liberando os recursos alocados no servidor e encerrando a conexão.

O exemplo evidencia o uso eficiente do protocolo IMAP na automação de decisões baseadas em mensagens de e-mail, com ênfase em segurança, controle de fluxo e não destrutividade. A abordagem é especialmente útil em ambientes corporativos que demandam **respostas automatizadas a comandos recebidos via e-mail**, integrando comunicação assíncrona a processos formais de negócio.

Esse modelo é útil em processos administrativos, financeiros e operacionais onde determinadas ações precisam de autorização eletrônica. O monitoramento

baseado em remetente ou assunto permite decisões rápidas e integração com sistemas internos como ERPs e plataformas de workflow.

4.5 Caso de uso automação de confirmação médica via e-mail: monitoramento IMAP com resposta SMTP

Nas rotinas administrativas de clínicas, laboratórios e instituições de saúde, é comum que setores de agendamento recebam diariamente solicitações de exames por e-mail, especialmente enviadas por convênios ou médicos parceiros. Tais mensagens geralmente seguem um padrão reconhecível, como a presença do termo “Solicitação Exame” no campo de assunto. O fluxo tradicional envolve a abertura manual de cada e-mail, verificação da agenda, seleção de um horário disponível e envio de resposta ao remetente. Esse processo, embora repetitivo, é crítico, pois envolve agendamentos sensíveis ao tempo e à disponibilidade da equipe médica e de equipamentos.

O atraso no processamento dessas solicitações pode ocasionar subutilização da agenda, falhas na confirmação dos exames e até mesmo retrabalho, uma vez que outros canais podem ser utilizados como alternativa caso o e-mail não seja respondido com a devida agilidade. O cenário torna-se ideal para a automação robótica de processos, especialmente com a utilização de agentes de software capazes de monitorar caixas de entrada, interpretar solicitações estruturadas, consultar serviços auxiliares e responder proativamente. Ao integrar os protocolos IMAP e SMTP em uma única aplicação, torna-se possível fechar um ciclo completo de comunicação automatizada, sem intervenção humana, com rastreabilidade e escalabilidade.

O exemplo a seguir apresenta um agente de automação construído em Python, que realiza a leitura de mensagens recebidas por meio do protocolo IMAP, identificando e-mails não lidos que contenham a expressão “Solicitacao Exame” no campo de assunto. Uma vez identificada uma mensagem relevante, o agente simula a consulta a uma agenda médica, identifica o próximo horário útil disponível e, em seguida, utiliza o protocolo SMTP para enviar uma resposta ao remetente original confirmando o agendamento. O fluxo visa demonstrar, de forma prática, a aplicabilidade da integração entre protocolos de correio eletrônico e lógica de

negócios, com potencial direto de adoção em ambientes clínicos, jurídicos, administrativos e corporativos em geral.

```
import os, ssl, imaplib, smtplib from datetime
import datetime, timedelta from email.message
import EmailMessage from email.header
import decode_header from email i
import message_from_bytes from dotenv
import load_dotenv

load_dotenv()

IMAP_HOST = "imap.gmail.com" SMTP_HOST = "smtp.gmail.com"
IMAP_PORT = 993 SMTP_PORT = 465
USER = os.getenv("GMAIL_USER") PASS = os.getenv("GMAIL_PASS")

def connect_imap():
     imap = imaplib.IMAP4_SSL(IMAP_HOST, IMAP_PORT)
     imap.login(USER, PASS)
     return imap

def fetch_new_requests(imap):
     imap.select("INBOX")
     status, ids = imap.search(None, '(UNSEEN SUBJECT
"Solicitacao Exame")')
     return ids[0].split()

def parse_message(imap, msg_id):
     status, data = imap.fetch(msg_id, "(RFC822)")
     msg = message_from_bytes(data[0][1])
     subject = decode_header(msg["Subject"])[0][0].decode()
     if isinstance(decode_header(msg["Subject"])[0][0], bytes)
     else msg["Subject"] sender = msg.get("From")
        return sender, subjetct

def next_available_slot():
     now = datetime.now()
     slot = now.replace(hour=8, minute=0, second=0,
microsecond=0)
     while slot < now or slot.weekday() >= 5:
        slot += timedelta(days=1)
        return slot.strftime("%d/%m/%Y %H:%M")



def send_confirmation(recipient, slot):
     msg = EmailMessage()
     msg["Subject"] = "Confirmação de  Exame"
     msg["From"] = USER
```

```
    msg["To"] = recipient msg.set_content(f"Seu exame foi
agendado para {slot}. Favor comparecer com 15 minutos de
antecedência.")
    context = ssl.create_default_context()

    with smtplib.SMTP_SSL(SMTP_HOST, SMTP_PORT,
context=context) as server:
        server.login(USER, PASS)
        server.send_message(msg)


def mark_as_seen(imap, msg_id):
    imap.store(msg_id, '+FLAGS', '\Seen')

def main():
    imap = connect_imap()
    try:
        ids = fetch_new_requests(imap)
        for msg_id in ids:
            sender, _ = parse_message(imap, msg_id)
            slot = next_available_slot()
            send_confirmation(sender, slot)
            mark_as_seen(imap, msg_id)
    finally:
        imap.logout()

if name == "main": main()
```

Código-fonte 11 - Exemplo de Caso Integrado
Fonte: Elaborado pelo autor (2025)

A estrutura do programa segue o modelo clássico de separação de responsabilidades, distribuindo a lógica de conexão, extração, decisão e resposta em funções bem definidas. Inicialmente, o script utiliza o pacote dotenv para carregar as credenciais do Gmail de um arquivo .env, prática recomendada para evitar a exposição de dados sensíveis. Em seguida, a função connect_imap realiza a conexão segura com o servidor IMAP do Gmail, autenticando o agente com os dados fornecidos.

A função fetch_new_requests é responsável por filtrar apenas as mensagens não lidas (com a flag UNSEEN) e cujo assunto contenha a expressão específica definida como critério de entrada. Essa operação utiliza a linguagem de busca do IMAP, que permite combinações lógicas eficientes. Os identificadores das mensagens retornadas são então passados para a função parse_message, que realiza a leitura

completa do conteúdo da mensagem, extrai o campo From e decodifica corretamente o campo Subject, considerando que este pode estar codificado com esquemas MIME.

Para simular a consulta de disponibilidade, a função next_available_slot gera a próxima data e hora úteis, assumindo funcionamento em horário comercial e exclusão de fins de semana. Essa lógica é implementada com auxílio da biblioteca datetime, sendo facilmente substituível por consultas reais a agendas via APIs RESTful ou banco de dados. A escolha de manter a lógica embarcada visa facilitar a replicabilidade do exemplo.

A resposta automatizada é então gerada pela função send_confirmation, que constrói um objeto EmailMessage, preenche os campos essenciais e envia a mensagem via SMTP sobre SSL, utilizando o servidor do Gmail. A operação de envio é encapsulada em um contexto de execução segura, que garante o encerramento da conexão mesmo em caso de exceções. Por fim, após o processamento da mensagem, a função mark_as_seen altera o status da mesma, garantindo que não seja reprocessada em execuções futuras do agente.

A função principal main orquestra a execução linear de todas as etapas, iniciando pela conexão, processamento das mensagens e encerramento da sessão IMAP. O código pode ser executado sob demanda ou incorporado a sistemas de agendamento com cron ou task scheduler, permitindo a execução periódica. Trata-se de um exemplo concreto de como protocolos de e-mail podem ser utilizados para orquestrar microprocessos organizacionais com alto impacto na qualidade do serviço prestado. A modelagem modular do agente ainda permite sua adaptação para diversos domínios, como atendimento jurídico, suporte técnico e logística hospitalar.

Ao final desta fase, em Assets, você encontrará outro código completo em Python, chamado “gmail_imap_rpa.py”. Este código serve para ler uma pasta de entrada de e-mail e, por meio da aplicação de filtros, definir quais e-mails corretos devem ser lidos.

CONCLUSÃO

Este capítulo explorou dois eixos tecnológicos centrais para a automação inteligente de processos: o consumo de serviços disponibilizados por meio de APIs REST e a comunicação automatizada via correio eletrônico utilizando os protocolos SMTP, POP3 e IMAP. A abordagem adotada privilegiou o ponto de vista da automação orientada por software, com ênfase em agentes que operam de forma autônoma e segura. Com isso, estabeleceu-se um alicerce técnico sólido que permite a integração de múltiplos sistemas, a resposta a eventos externos e a execução de fluxos de trabalho complexos sem necessidade de intervenção humana direta.

No domínio das APIs REST, foi demonstrado como as operações CRUD podem ser realizadas sobre HTTP com pleno controle sobre os métodos, cabeçalhos, carga útil e códigos de resposta. A autenticação por meio de tokens JWT foi apresentada como uma solução segura, escalável e alinhada com o paradigma *stateless*, fundamental em arquiteturas distribuídas. As práticas descritas neste capítulo fornecem a base para o desenvolvimento de sistemas conectados, interoperáveis e prontos para suportar a lógica de decisão baseada em regras ou IA.

Quanto ao uso de e-mails como canal de integração e ativação de processos, observou-se que o monitoramento automatizado de mensagens permite capturar solicitações, interpretar seu conteúdo e disparar ações programadas com alta confiabilidade. A leitura de mensagens por POP3 e IMAP, bem como o envio estruturado por SMTP, constituem mecanismos de automação versáteis, especialmente úteis em contextos onde o correio eletrônico ainda é o meio preferencial de comunicação entre usuários e sistemas. Os exemplos apresentados demonstraram como agentes automatizados podem atuar de forma reativa e coordenada com outras aplicações.

Por fim, o domínio combinado dessas duas capacidades resulta em soluções tecnicamente sólidas e operacionalmente eficazes. A automação de processos ganha corpo e inteligência quando agentes são capazes de consumir dados de APIs, correlacionar esses dados com entradas oriundas de mensagens eletrônicas e executar decisões autônomas com base em políticas e algoritmos bem definidos. Trata-se de um conjunto de competências essenciais para profissionais e

pesquisadores que desenvolvem sistemas modernos de automação cognitiva, capazes de interagir com o ecossistema digital de maneira proativa, confiável e auditável.

FIAP

REFERÊNCIAS

HALEEM, Abid *et al.* Hyperautomation for the enhancement of automation in industries. **Sensors International**, v. 2, p. 100124, 2021.

MADAKAM, Somayya; HOLMUKHE, Rajesh M.; REVULAGADDA, Rajeev K. The next generation intelligent automation: hyperautomation. **JISTEM-Journal of Information Systems and Technology Management**, v. 19, p. e202219009, 2022.

PILGRIM, Mark; WILLISON, Simon. **Dive into python**. 3. ed. New York: Apress, 2009.

FIAP